

Yes. The cleanest way to do this is to treat the assignment as a small controlled experiment rather than “run JMeter and screenshot whatever appears.”

Below is the procedure I would follow from an **empty JMeter window all the way to the final ZIP**. Wherever I do not explicitly tell you to change a field, leave JMeter's default value unchanged.

The assignment requires five specific endpoints, 50- and 100-thread profiles, 100-second ramp-up, Recording Controllers, the HTTP(S) Test Script Recorder, Duration Assertions, View Results Tree, Aggregate Report, Summary Report, CSV results, and an HTML dashboard.

---

## 0. Create your assignment directory first

Do **not** save files inside the JMeter `bin` directory.

Suppose your student ID is:

```
2205001
```

Create:

```
CSE214_JMeter_2205001/  
|  
├─ test-plan/  
|  
├─ results/  
|   ├─ 50-thread/  
|   └─ 100-thread/  
|  
├─ html-report/  
|   ├─ 50-thread/  
|   └─ 100-thread/  
|  
├─ report/  
|   ├─ figures/  
|   └─ tables/  
|  
├─ screenshots/  
|  
└─ submission/
```

Eventually I want it to look like:

```
CSE214_JMeter_2205001/
|
├─ test-plan/
|   └─ 2205001_testPlan.jmx
|
├─ results/
|   ├── 50-thread/
|   |   ├── 2205001_result_50.csv
|   |   ├── 2205001_aggregate_50.csv
|   |   └─ 2205001_summary_50.csv
|   |
|   └─ 100-thread/
|       ├── 2205001_result_100.csv
|       ├── 2205001_aggregate_100.csv
|       └─ 2205001_summary_100.csv
|
├─ html-report/
|   ├── 50-thread/
|   |   └─ [JMeter-generated HTML files]
|   └─ 100-thread/
|       └─ [JMeter-generated HTML files]
|
├─ report/
|   ├── figures/
|   ├── 2205001_Assessment.tex
|   └─ 2205001_Assessment.pdf
|
├─ screenshots/
|
└─ submission/
    ├── 2205001_Assessment.pdf
    ├── 2205001_testPlan.jmx
    ├── 2205001_result_50.csv
    ├── 2205001_result_100.csv
    └─ 2205001_html.zip
```

Your assignment uses singular names such as `yourID result.csv` and `yourID html.zip`, while simultaneously requiring two separate load profiles. I would **preserve both raw CSVs** rather than throwing one away. You can package both HTML dashboards inside one `yourID_html.zip`.

# 1. What you actually need to import/install

You do **not** need:

- third-party JMeter plugins;
- an external JMX template;
- an SSL certificate;
- a CSV dataset;
- a database;
- browser extensions.

The assignment server is plain HTTP, so HTTPS certificate configuration is explicitly unnecessary.

You need:

```
Apache JMeter
A browser – Firefox is convenient for proxy recording
Your LaTeX environment
```

You should already have JMeter working.

When you start JMeter, start with an empty:

```
Test Plan
```

## 2. The final JMeter tree you are trying to construct

Your final `.jmx` should approximately look like this:

```

Test Plan: CSE214 LoadLab Performance Assessment
|
|— Thread Group - LOW LOAD - 50
|   |
|   |— HTTP Request Defaults - LoadLab
|   |— HTTP Cookie Manager - Sessions
|   |
|   |— Recording Controller - HOME
|       |— HOME - GET /
|           |— Duration Assertion - T ms
|       |
|       |— Recording Controller - NOTICES
|           |— NOTICES - GET /notices
|               |— Duration Assertion - T ms
|       |
|       |— Recording Controller - COURSES
|           |— COURSES - GET /courses
|               |— Duration Assertion - T ms
|       |
|       |— Recording Controller - LOGIN
|           |— LOGIN - POST /login
|               |— Duration Assertion - T ms
|       |
|       |— Recording Controller - DOWNLOAD
|           |— DOWNLOAD - GET /api/download/256
|               |— Duration Assertion - T ms
|
|— Thread Group - HIGH LOAD - 100
|   |— [duplicate of everything above]
|
|— View Results Tree
|— Aggregate Report
|— Summary Report
|— Simple Data Writer
|
|— HTTP(S) Test Script Recorder - LoadLab Recorder
    
```

A Recording Controller is specifically intended as the destination into which JMeter's recorder creates samples. During actual execution it behaves essentially as an organizational controller. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

### 3. Create the Test Plan itself

Click:

```
Test Plan
```

On the right:

```
Name:
CSE214 LoadLab Performance Assessment
```

Leave:

```
Functional Testing
```

**unchecked.**

Do not turn Functional Testing on for load testing; it can cause response data to be retained and increase overhead. JMeter's documentation similarly recommends keeping it off for stress/load testing. ([Apache JMeter](https://jmeter.apache.org/usermanual/test_plan.html?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/test\\_plan.html?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/test_plan.html?utm_source=chatgpt.com)))

Save immediately:

```
File
→ Save Test Plan As...
```

Save to:

```
CSE214_JMeter_2205001/test-plan/2205001_testPlan.jmx
```

From now on, press `Ctrl+S` frequently.

## 4. Create the 50-thread group

Right-click:

```
Test Plan
→ Add
→ Threads (Users)
→ Thread Group
```

JMeter defines a Thread Group as the collection of simulated users. The ramp-up is how long JMeter takes to start those users. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/component\\_reference?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/component_reference?utm_source=chatgpt.com)))

Configure it exactly:

| Field                               | Value                        |
|-------------------------------------|------------------------------|
| Name                                | Thread Group - LOW LOAD - 50 |
| Action after Sampler error          | Continue                     |
| Number of Threads (users)           | 50                           |
| Ramp-up period (seconds)            | 100                          |
| Loop Count                          | 1                            |
| Infinite/Forever                    | unchecked                    |
| Delay Thread creation until needed  | leave default                |
| Specify Thread lifetime / Scheduler | unchecked                    |

Keep `Continue` .

This matters because some requests are intentionally going to fail the Duration Assertion. You do **not** want one assertion failure to terminate the thread and prevent `/courses` , `/login` , etc. from executing.

Your assignment explicitly asks for:

```
Low load: 50 threads
High load: 100 threads
Ramp-up: 100 seconds
```

### A subtle point about ramp-up

For 50 threads over 100 seconds:

$$\frac{50}{100} = 0.5 \text{ new threads/s}$$

or roughly one new user every two seconds.

For 100 threads:

$$\frac{100}{100} = 1 \text{ new thread/s.}$$

Therefore “100 threads” does **not** mean that JMeter immediately fires 100 users simultaneously.

Follow the assignment exactly anyway.

## 5. Add HTTP Request Defaults

Right-click:

```
Thread Group - LOW LOAD - 50
→ Add
→ Config Element
→ HTTP Request Defaults
```

Set:

| Field             | Value                           |
|-------------------|---------------------------------|
| Name              | HTTP Request Defaults - LoadLab |
| Protocol          | http                            |
| Server Name or IP | 103.94.135.91                   |
| Port Number       | 8080                            |
| Path              | blank                           |
| Content Encoding  | blank                           |
| Implementation    | HttpClient4 if selectable       |
| Connect Timeout   | blank                           |
| Response Timeout  | blank                           |

Do **not** type:

```
http://103.94.135.91
```

into Server Name.

It should only be:

```
103.94.135.91
```

JMeter's documentation explicitly says the server field should not contain the `http://` prefix. HTTP Request Defaults exists so every HTTP sampler can inherit the same server/port instead of repeating it. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

After doing this, individual requests only need paths such as:

```
/
/notices
/courses
/login
/api/download/256
```

## 6. Add the HTTP Cookie Manager

Right-click:

```
Thread Group - LOW LOAD - 50
→ Add
→ Config Element
→ HTTP Cookie Manager
```

Set:

```
Name:
HTTP Cookie Manager - Sessions
```

Set:

```
Clear cookies each iteration:  
checked
```

Leave the manually configured cookie table empty.

Leave Cookie Policy at its default unless the application behaves incorrectly.

Why do we need this?

Because your `/login` endpoint is explicitly supposed to exercise session-cookie behavior.

JMeter's Cookie Manager behaves much like a browser cookie jar: cookies returned by the server are stored and later resent, and **each JMeter thread has its own cookie storage**.

([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

That is exactly what you want for simulated independent users.

---

## 7. Add the five Recording Controllers

Right-click your 50-thread group five times:

```
Thread Group - LOW LOAD - 50  
→ Add  
→ Logic Controller  
→ Recording Controller
```

Rename them:

```
Recording Controller - HOME  
Recording Controller - NOTICES  
Recording Controller - COURSES  
Recording Controller - LOGIN  
Recording Controller - DOWNLOAD
```

Do not put requests inside them manually yet.

The recorder will generate them.

---

## 8. Add the HTTP(S) Test Script Recorder

Right-click:

```
Test Plan
→ Add
→ Non-Test Elements
→ HTTP(S) Test Script Recorder
```

The recorder works by becoming a temporary HTTP proxy between your browser and the application. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

Configure it as follows.

## Basic settings

```
Name:
HTTP(S) Test Script Recorder - LoadLab
```

```
Port:
8888
```

8888 is JMeter's default recorder port. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

For:

```
Target Controller
```

select the particular Recording Controller you want to record into when possible.

For example, when recording :

```
Thread Group - LOW LOAD - 50
> Recording Controller - HOME
```

If your JMeter version only conveniently shows:

```
Use Recording Controller
```

that's okay. Let it record into the first Recording Controller and then drag the resulting sampler into the appropriate Recording Controller afterward.

## Grouping

---

Set:

```
Grouping:  
Do not group samplers
```

You want individual HTTP samplers, not extra transaction/simple-controller structures.

## Recorder options

---

Use:

```
Capture HTTP Headers:  
unchecked
```

for a clean script.

```
Add Assertions:  
unchecked
```

because you'll add the required Duration Assertions yourself.

```
Regex Matching:  
unchecked
```

```
Redirect Automatically:  
unchecked
```

```
Follow Redirects:  
checked
```

```
Use Keep-Alive:
checked
```

```
Retrieve all Embedded Resources:
unchecked
```

The latter is important because otherwise one HTML page could suddenly produce requests for CSS, images, JavaScript, fonts, etc., contaminating your five-endpoint experiment.

## 9. Add a recording filter

Without filters, the recorder may capture:

```
/favicon.ico
/style.css
/main.js
logo.png
fonts
browser background requests
...
```

JMeter's recorder supports URL include/exclude regular expressions precisely for this reason.

([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

Under:

```
Patterns to Include
```

add:

```
103\.94\.135\.91:8080/.*
```

Under:

```
Patterns to Exclude
```

you can add:

```
.*\.(css|js|png|jpg|jpeg|gif|svg|ico|woff|woff2) (\?.*)?
```

You still need to inspect what is recorded afterward.

Do not blindly trust filtering.

---

## 10. Add your listeners now

Right-click:

```
Test Plan
→ Add
→ Listener
```

Add:

```
View Results Tree
Aggregate Report
Summary Report
Simple Data Writer
```

Rename them if desired:

```
View Results Tree - DEBUG ONLY
Aggregate Report - METRICS
Summary Report - METRICS
Simple Data Writer - RAW CSV
```

The assignment explicitly requires the first three.

`Simple Data Writer` is my additional recommendation because it gives you a clean low-overhead place to write raw result CSVs.

JMeter specifically describes Simple Data Writer as an efficient result writer without GUI visualization overhead. ([jmeter.apache.org](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

---

## 11. Configure the browser proxy

I recommend Firefox because it can have its own proxy configuration independently from your whole operating system.

Go to Firefox:

```
Settings
→ search "proxy"
→ Network Settings
→ Settings...
```

Choose:

```
Manual proxy configuration
```

Set:

```
HTTP Proxy:
127.0.0.1
```

```
Port:
8888
```

Make sure:

```
103.94.135.91
```

is **not** listed under "No proxy for".

Since your target application is HTTP rather than HTTPS, you do not need to install JMeter's temporary CA certificate.

Preferably open a **Private/Incognito window** for recording. JMeter itself recommends using a private browsing session when recording because old cookies and browser state can interfere. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

## 12. Start the recorder

Return to JMeter.

Select:

```
HTTP(S) Test Script Recorder - LoadLab
```

Click:

```
Start
```

JMeter should indicate that the proxy is running.

Now your Firefox traffic should flow:

```
Firefox
  ↓
127.0.0.1:8888
  ↓
JMeter Recorder
  ↓
103.94.135.91:8080
```

JMeter's official instructions likewise require configuring the browser to use the machine running JMeter and the recorder port. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

---

## 13. Record endpoint 1: Home Page

Set Target Controller to:

```
Recording Controller - HOME
```

In Firefox go to:

```
http://103.94.135.91:8080/
```

Return to JMeter.

You should see an HTTP sampler appear.

Keep the request corresponding to:

GET /

Remove unrelated requests.

Rename the sampler:

HOME - GET /

Open it and verify:

| Field                       | Desired value |
|-----------------------------|---------------|
| Protocol                    | blank         |
| Server                      | blank         |
| Port                        | blank         |
| Method                      | GET           |
| Path                        | /             |
| Redirect Automatically      | unchecked     |
| Follow Redirects            | checked       |
| Use KeepAlive               | checked       |
| Retrieve Embedded Resources | unchecked     |

Protocol/server/port should ideally be blank because they are inherited from HTTP Request Defaults.

The assignment describes this endpoint as the baseline page with roughly 0–30 ms server latency and a 2 KB response.

---

## 14. Record Notice Board

Change Target Controller to:

```
Recording Controller - NOTICES
```

Browse:

```
http://103.94.135.91:8080/notices
```

Keep only:

```
GET /notices
```

Rename it:

```
NOTICES - GET /notices
```

Verify:

```
Method = GET  
Path   = /notices
```

Remove junk requests.

The assignment says this endpoint intentionally has a random uniform delay of approximately 100–400 ms and a 7 KB response.

That randomness is important later in your analysis.

---

## 15. Record Course Catalogue

Target:

```
Recording Controller - COURSES
```

Browse:

```
http://103.94.135.91:8080/courses
```

Keep:

```
GET /courses
```

Rename:

```
COURSES - GET /courses
```

Verify:

```
Method = GET  
Path   = /courses
```

The assignment says this endpoint has a baseline 150–250 ms component plus CPU work whose latency scales under load, making this endpoint particularly interesting when comparing 50 vs 100 users.

---

## 16. Record Login properly

This one needs more attention.

Target:

```
Recording Controller - LOGIN
```

Open whatever page contains the login form.

Enter:

```
username:  
student
```

```
password:
student123
```

Submit the form.

You may record something like:

```
GET /login
POST /login
GET /
```

or similar.

The assignment specifically wants:

```
POST /login
```

so keep the POST sampler.

Rename it:

```
LOGIN - POST /login
```

Delete unrelated navigation samplers.

Open the sampler.

Verify:

```
Method:
POST
```

```
Path:
/login
```

Under:

```
Parameters
```

you should have:

| Name     | Value      |
|----------|------------|
| username | student    |
| password | student123 |

Keep encoding enabled.

Do **not** enable:

```
Use multipart/form-data
```

This should be ordinary:

```
application/x-www-form-urlencoded
```

which is what your assignment requires.

Also use:

```
Redirect Automatically:
unchecked
```

```
Follow Redirects:
checked
```

This choice is useful because JMeter's `Follow Redirects` mode makes redirect responses visible as additional/sub-samples, whereas `Redirect Automatically` hides them inside the HTTP implementation and is not intended for POST in this context. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

So later in View Results Tree you can demonstrate:

```
POST /login
→ HTTP 303
→ redirected request
```

which is exactly one of the behaviors your assignment asks you to investigate.

---

## 17. Record the bandwidth endpoint

The assignment permits either:

```
GET /api/download/256
```

or:

```
POST /api/upload
```

but its provided table specifically uses `/api/download/256`.

Use the download endpoint.

Target:

```
Recording Controller - DOWNLOAD
```

Navigate:

```
http://103.94.135.91:8080/api/download/256
```

Keep:

```
GET /api/download/256
```

Rename:

```
DOWNLOAD - GET /api/download/256
```

Verify:

```
Method = GET  
Path   = /api/download/256
```

---

## 18. Stop recording immediately

Return to:

```
HTTP(S) Test Script Recorder
```

Click:

```
Stop
```

Then return Firefox's proxy configuration to:

```
No proxy
```

or your normal settings.

Do not forget this.

Otherwise you will later wonder why your browser “lost internet” when JMeter is closed.

---

## 19. Clean the recorded script

At this point, your LOW LOAD Thread Group should contain **exactly five relevant HTTP samplers**.

Delete things such as:

```
GET /favicon.ico
GET /style.css
GET /main.js
GET /some-image.png
GET /login      ← if merely navigation
duplicate redirect requests
browser background traffic
```

Keep:

```
HOME - GET /
NOTICES - GET /notices
COURSES - GET /courses
LOGIN - POST /login
DOWNLOAD - GET /api/download/256
```

Do not blindly delete an automatically generated Header Manager attached to `/login` until you verify login still works.

If it contains only irrelevant browser headers such as:

```
User-Agent
Accept-Language
Sec-Fetch-...
```

it can generally be removed.

If it contains something essential to the POST such as:

```
Content-Type: application/x-www-form-urlencoded
```

you can either retain it or verify JMeter generates the appropriate content type from the POST parameters.

The principle is:

*Your JMeter script should represent the application protocol, not reproduce every accidental detail of your browser.*

## 20. Do a functional validation before any load test

This is where View Results Tree belongs.

JMeter's documentation explicitly warns that **View Results Tree should not be used for real load testing**, because it consumes substantial CPU and memory. It is intended for validation/debugging. ([jmeter.apache.org](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

Temporarily change LOW LOAD to:

```
Number of Threads:
1

Ramp-up:
1

Loop Count:
1
```

Enable:

```
View Results Tree
```

Run:

```
Run
→ Start
```

or click the green Start triangle.

## 21. Validate every request

In View Results Tree inspect all five.

### Home

Expect a successful HTTP response.

Verify:

```
Request URL
Method
Response Code
Response Data
```

## Notices

---

Verify HTTP success and sensible response body.

## Courses

---

Same.

## Login

---

This is important.

Open:

```
Request
```

and ensure the POST contains:

```
username=student
password=student123
```

Inspect:

```
Response Headers
```

and redirect sub-results.

You are looking for evidence of:

```
303 redirect
```

and possibly:

```
Set-Cookie
```

Because Cookie Manager is present, the cookie can then be reused by the thread. JMeter's Cookie Manager is specifically designed to automatically store server cookies and send them with later applicable requests. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

## Download

---

Verify the response arrives successfully and has substantial response bytes.

---

## 22. If anything is incorrect, fix it now

Do not continue until:

```
GET /
GET /notices
GET /courses
POST /login
GET /api/download/256
```

all function correctly.

Typical problems:

```
Wrong host
Wrong port
/login accidentally GET instead of POST
credentials in Body Data incorrectly
browser junk samplers still present
duplicate redirect sampler
Cookie Manager missing
```

The correct time to debug these is with **one user**, not 100.

---

## 23. Restore your required Thread Group

Set it back to:

```
Number of Threads:
50

Ramp-up:
100

Loop:
1
```

---

## 24. Add Duration Assertions

Now add one to **each** of your five HTTP requests.

For example:

```
HOME - GET /
→ right-click
→ Add
→ Assertions
→ Duration Assertion
```

Do this five times.

Initially name them:

```
Duration Assertion - 200 ms
```

Set:

```
Duration in Milliseconds:
200
```

JMeter defines a Duration Assertion very simply: if the response takes longer than the specified number of milliseconds, the sample is marked as failed. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) ([https://jmeter.apache.org/usermanual/component\\_reference.html](https://jmeter.apache.org/usermanual/component_reference.html)))

So with:

$$T = 200 \text{ ms}$$

you effectively have:

$$\text{success}(t) = \begin{cases} 1, & t \leq 200 \\ 0, & t > 200. \end{cases}$$

## 25. Why I would initially try 200 ms

Your assignment itself gives 200 ms as an example and requires you to choose a threshold that produces both successful and failed events.

The provided endpoint characteristics make 200 ms a reasonable **initial calibration value**:

|         |             |                                 |
|---------|-------------|---------------------------------|
| Home    | 0-30 ms     | → likely passes                 |
| Notices | 100-400 ms  | → some likely pass, some fail   |
| Courses | 150-250+ ms | → likely mixed / load-sensitive |

That is exactly what we want.

But **do not write in your report that 200 ms was definitely appropriate before observing your data.**

Instead:

1. Start with 200 ms.
2. Run LOW LOAD.
3. Inspect Error %.
4. If everything passes, lower the threshold.
5. If everything fails, increase it.
6. Repeat until you obtain meaningful pass/fail behavior.
7. Once chosen, **keep the same threshold for both 50 and 100 threads.**

Do not use different thresholds for the two load profiles.

Then the error-rate comparison remains meaningful.

## 26. Important: assertion failure ≠ HTTP

# failure

Suppose:

```
HTTP response:  
200 OK
```

but:

```
Elapsed:  
327 ms
```

and:

```
Duration threshold:  
200 ms
```

Then JMeter marks that sample failed.

This is conceptually:

$$327 > 200 \Rightarrow \text{SLA violation} \Rightarrow \text{failed JMeter sample.}$$

The application still returned a legitimate HTTP response.

That distinction should appear in your report.

It is one of the most important analytical observations in the whole assignment.

---

## 27. Verify the assertion visually once

Use:

```
View Results Tree
```

for a small/debug run.

Find:

```
a green successful sample
```

and:

```
a red assertion-failed sample
```

Save screenshots of both to:

```
screenshots/assertion-pass.png
screenshots/assertion-fail.png
```

These will also be useful during your video.

Then disable View Results Tree before the official performance measurement.

Right-click:

```
View Results Tree - DEBUG ONLY
→ Disable
```

Keep it physically inside the test plan because the assignment requires the listener, but don't let it distort your load measurements.

## 28. Add/verify Simple Data Writer

Select:

```
Simple Data Writer - RAW CSV
```

Before the 50-thread run, set its Filename to:

```
.../results/50-thread/2205001_result_50.csv
```

Use the Browse button rather than typing a long path if convenient.

Click:

```
Configure
```

and keep CSV output.

Do **not** aggressively disable result fields.

The HTML dashboard requires items such as labels, bytes, latency, response code/message, etc.; JMeter's default CSV settings contain the needed fields. ([Apache JMeter](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/generating-dashboard?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com)))

Simple Data Writer will create your raw per-sample data.

This CSV is different from the CSV produced by the Aggregate Report's **Save Table Data** button.

Think:

```
Simple Data Writer
  ↓
raw samples
```

versus:

```
Aggregate Report
  ↓
calculated statistics per endpoint
```

## 29. Duplicate the complete 50-thread group

Only now, after everything works:

Right-click:

```
Thread Group - LOW LOAD - 50
→ Duplicate
```

Rename the duplicate:

```
Thread Group - HIGH LOAD - 100
```

Change exactly one experimental factor:

```
Number of Threads:  
50 → 100
```

Keep:

```
Ramp-up = 100  
Loop Count = 1  
Duration threshold = same  
Requests = same  
Credentials = same  
Endpoints = same
```

This is good experimental design.

Conceptually:

independent variable = configured user load.

Everything else should remain controlled.

---

## 30. How to enable/disable the thread groups

For LOW LOAD:

```
Thread Group - LOW LOAD - 50  
    ENABLED  
  
Thread Group - HIGH LOAD - 100  
    DISABLED
```

For HIGH LOAD:

```
Thread Group - LOW LOAD - 50
    DISABLED

Thread Group - HIGH LOAD - 100
    ENABLED
```

Never accidentally run both together.

The shared server instructions explicitly say not to exceed 100 threads.

---

## 31. Official 50-thread run

Configure:

```
LOW  = enabled
HIGH = disabled
```

Listeners:

```
View Results Tree = disabled
Aggregate Report   = enabled
Summary Report     = enabled
Simple Data Writer = enabled
```

Set Simple Data Writer Filename:

```
results/50-thread/2205001_result_50.csv
```

Before starting, make sure this file doesn't already contain an earlier experimental run.

Then:

```
Run
→ Clear All
```

Then:

```
Run
→ Start
```

Allow the test to finish.

Do not click Stop unless necessary.

Wait until active thread count returns to zero.

## 32. Save the 50-thread Aggregate Report

Open:

```
Aggregate Report - METRICS
```

You should have rows such as:

```
HOME - GET /
NOTICES - GET /notices
COURSES - GET /courses
LOGIN - POST /login
DOWNLOAD - GET /api/download/256
TOTAL
```

The Aggregate Report provides:

```
Samples
Average
Min
Max
Error %
Throughput
Received KB/sec
Sent KB/sec
```

among other fields, and its timing units are milliseconds. ([jmeter.apache.org](https://jmeter.apache.org/usermanual/component_reference) (https://jmeter.apache.org/usermanual/component\_reference))

Those fields line up almost perfectly with your assignment requirements.

Click:

```
Save Table Data
```

save:

```
results/50-thread/2205001_aggregate_50.csv
```

Do the same from Summary Report if desired:

```
results/50-thread/2205001_summary_50.csv
```

Save Table Data is intended specifically for exporting the calculated Aggregate/Summary table.

## 33. Check your raw CSV immediately

Check:

```
results/50-thread/2205001_result_50.csv
```

It should be nonempty.

It should contain many rows — roughly hundreds because each virtual user executes multiple samplers.

With 50 threads and five requests:

$$50 \times 5 = 250$$

main sampler executions are expected in the simplest case.

Redirects/subsamples can make the underlying structure somewhat more complicated.

## 34. Run the 100-thread experiment

Now:

```
LOW  = disabled  
HIGH = enabled
```

Change Simple Data Writer Filename:

```
results/100-thread/2205001_result_100.csv
```

Do:

```
Run  
→ Clear All
```

Then:

```
Run  
→ Start
```

Wait for completion.

Export Aggregate Report:

```
results/100-thread/2205001_aggregate_100.csv
```

and optionally Summary Report:

```
results/100-thread/2205001_summary_100.csv
```

Now you have both experimental datasets.

---

## 35. Do not overwrite the raw CSVs

Your two most important raw files are now:

```
2205001_result_50.csv  
2205001_result_100.csv
```

Treat them as read-only experimental evidence.

If you rerun a test, save:

```
2205001_result_50_rerun.csv
```

until you decide which run is your official run.

Do not casually overwrite your good data.

---

## 36. Generate the 50-thread HTML dashboard

JMeter officially supports generating the dashboard from an existing CSV through:

```
Tools  
→ Generate HTML Report
```

([Apache JMeter](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/generating-dashboard?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com)))

Choose:

### Results file

```
CSE214_JMeter_2205001/  
results/50-thread/  
2205001_result_50.csv
```

### user.properties file

Select JMeter's:

```
apache-jmeter-.../bin/user.properties
```

You do not need to modify it.

### Output directory

Select:

```
CSE214_JMeter_2205001/  
html-report/50-thread/
```

The output folder should be empty before generation.

Click:

```
Generate report
```

When finished, open:

```
html-report/50-thread/index.html
```

in a browser.

The generated dashboard includes statistics and graphs such as errors, response times, active threads, throughput and byte throughput. ([Apache JMeter](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/generating-dashboard?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com)))

---

## 37. Generate the 100-thread dashboard

Repeat with:

```
results/100-thread/2205001_result_100.csv
```

and output to:

```
html-report/100-thread/
```

Open:

```
html-report/100-thread/index.html
```

Verify it works.

---

## 38. Alternative command-line dashboard generation

You don't need this for the assignment, but it is useful to understand.

From JMeter:

```
jmeter -g 2205001_result_50.csv -o report_50
```

means:

```
-g = generate dashboard from existing log  
-o = output directory
```

JMeter also supports executing and generating the report in one operation:

```
jmeter -n \  
-t 2205001_testPlan.jmx \  
-l 2205001_result_50.csv \  
-e \  
-o report_50
```

The official documentation specifies both forms. ([Apache JMeter](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com) ([https://jmeter.apache.org/usermanual/generating-dashboard?utm\\_source=chatgpt.com](https://jmeter.apache.org/usermanual/generating-dashboard?utm_source=chatgpt.com)))

For your assignment, the GUI workflow is easier to demonstrate in the mandatory video.

---

## 39. ZIP the HTML reports

Do **not** take only:

```
index.html
```

The dashboard depends on CSS/JS/data files around it.

Create:

```
2205001_html/  
├─ 50-thread/  
│   └─ [complete dashboard]  
└─ 100-thread/  
    └─ [complete dashboard]
```

Zip that entire directory:

```
2205001_html.zip
```

Now opening either dashboard after extracting the ZIP should still work.

---

## 40. What files are configuration files?

There is really only one configuration artifact you need to submit:

```
2205001_testPlan.jmx
```

That `.jmx` file contains your:

```
Thread Groups  
Recording Controllers  
HTTP samplers  
Cookie Manager  
HTTP defaults  
Assertions  
Listeners  
Recorder configuration
```

The assignment explicitly asks for the exported JMeter Test Plan containing the Thread Groups, Recording Controllers, Assertions and listeners.

You do **not** need to submit:

```
jmeter.properties  
user.properties  
jmeter.log  
JMeter installation  
browser proxy configuration
```

unless your instructor separately requests them.

---

## 41. Which screenshots should you capture?

You don't need screenshots of everything.

Capture useful evidence:

```
01_full_test_plan_tree.png
02_thread_group_50_config.png
03_thread_group_100_config.png
04_http_request_defaults.png
05_recorder_config.png
06_login_post_parameters.png
07_duration_assertion.png
08_assertion_pass.png
09_assertion_fail.png
10_aggregate_50.png
11_aggregate_100.png
12_html_dashboard_50.png
13_html_dashboard_100.png
```

Put them in:

```
screenshots/
```

Then copy only the best ones into:

```
report/figures/
```

---

## 42. What your results tables should contain

Your assignment explicitly gives this table structure.

Your first report table should become:

| Endpoint          | Threads | Min (ms) | Max (ms) | Avg (ms) | Throughput | Error % |
|-------------------|---------|----------|----------|----------|------------|---------|
| /                 | 50      |          |          |          |            |         |
| /                 | 100     |          |          |          |            |         |
| /notices          | 50      |          |          |          |            |         |
| /notices          | 100     |          |          |          |            |         |
| /courses          | 50      |          |          |          |            |         |
| /courses          | 100     |          |          |          |            |         |
| /login            | 50      |          |          |          |            |         |
| /login            | 100     |          |          |          |            |         |
| /api/download/256 | 50      |          |          |          |            |         |
| /api/download/256 | 100     |          |          |          |            |         |

Use values from **Aggregate Report**.

Do not type numbers from memory.

### 43. Make a second table for bandwidth

The assignment asks for:

- Average Throughput;
- Average Data Reception Rate;
- Average Data Transmission Rate.

So use another table:

| Endpoint | Threads | Throughput (req/s) | Received KB/s | Sent KB/s |
|----------|---------|--------------------|---------------|-----------|
| Home     | 50      |                    |               |           |
| Home     | 100     |                    |               |           |
| Notices  | 50      |                    |               |           |
| Notices  | 100     |                    |               |           |
| Courses  | 50      |                    |               |           |
| Courses  | 100     |                    |               |           |
| Login    | 50      |                    |               |           |
| Login    | 100     |                    |               |           |
| Download | 50      |                    |               |           |
| Download | 100     |                    |               |           |

Aggregate Report exposes `Received KB/sec` and `Sent KB/sec` directly. ([jmeter.apache.org](https://jmeter.apache.org) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

## 44. Do some actual quantitative analysis

Don't merely reproduce JMeter's table.

For average response time, compute:

$$\Delta t = t_{100} - t_{50}.$$

More useful is percentage change:

$$\% \Delta t = \frac{t_{100} - t_{50}}{t_{50}} \times 100.$$

Example:

Suppose:

$$t_{50} = 190 \text{ ms}$$

and:

$$t_{100} = 260 \text{ ms}.$$

Then:

$$\% \Delta t = \frac{260 - 190}{190} \times 100 \approx 36.84\%.$$

Then write:

*The average response time increased by approximately 36.8% when the configured load increased from 50 to 100 threads.*

That's much better than:

*100 threads were slower.*

Do the same for throughput where useful:

$$\% \Delta X = \frac{X_{100} - X_{50}}{X_{50}} \times 100.$$

## 45. Interpret each endpoint differently

Do not analyze all five as if they perform the same work.

### Home /

This is your baseline.

The assignment describes it as having very low latency and a small 2 KB response.

Discuss whether it remains relatively stable under increased load.

### Notices /notices

The server intentionally introduces a random uniform delay between approximately:

$$100 \text{ ms} \leq t \leq 400 \text{ ms}.$$

Therefore:

*Variation is expected even under the same load profile.*

Don't attribute every timing difference to concurrency.

## Courses /courses

This is CPU-intensive and intended to scale in latency under user load.

This is probably the most interesting endpoint for:

$$t_{50} \quad \text{vs} \quad t_{100}.$$

If it becomes substantially slower at 100 threads, you can state that the observation is **consistent with increased CPU contention**.

Do not claim you directly measured server CPU usage—you didn't.

## Login /login

Discuss:

```
POST body
303 redirect
session cookie behavior
response time
assertion result
```

Be explicit that a login sample may include redirect processing when Follow Redirects is enabled.

JMeter documents that with Follow Redirects, the parent elapsed time can include the redirect chain. ([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference.html) (https://jmeter.apache.org/usermanual/component\_reference.html))

## Download /api/download/256

The most interesting metrics are:

```
Received KB/sec
Throughput
Response Time
```

This endpoint is specifically intended for data-transfer/bandwidth measurements.

---

## 46. Your report structure

I recommend:

Title Page

YouTube Demonstration Link

1. Introduction

2. Experimental Setup

2.1 Target Application

2.2 Endpoint Characteristics

2.3 JMeter Test Plan

2.4 Load Profiles

2.5 Duration Assertion

3. Test Methodology

3.1 HTTP Recording Procedure

3.2 Request Validation

3.3 50-Thread Experiment

3.4 100-Thread Experiment

3.5 Metric Collection

4. Experimental Results

4.1 Execution Time and Throughput

4.2 Bandwidth Measurements

4.3 Error and Assertion Results

5. Performance Analysis

5.1 Home Page

5.2 Notice Board

5.3 Course Catalogue

5.4 Authentication

5.5 Data Download

6. Duration Assertion Analysis

7. Limitations

8. Conclusion

Appendix

---

## 47. What to write in the Introduction

Keep it short.

Explain:

- what load testing measures;
- why JMeter is being used;
- what application you're testing;
- the purpose of comparing two workloads.

For example, conceptually:

```
The objective of this experiment is to evaluate the response-time,
throughput, bandwidth, and error characteristics of the LoadLab
application under two configured user loads. Apache JMeter is used
to record HTTP interactions and replay them with 50 and 100 virtual
users while maintaining a 100-second ramp-up period.
```

Then mention your five endpoints.

Do **not** spend two pages explaining what HTTP is.

---

## 48. Put the YouTube link near the beginning

This is mandatory.

The assignment says it must be prominently featured at the beginning of the report.

In LaTeX:

```
\section*{Video Demonstration}

The complete JMeter workflow and experiment demonstration is available at:

\url{https://youtu.be/YOUR_VIDEO_ID}
```

Do not bury it in the appendix.

---

## 49. Experimental Setup section

Include a configuration table such as:

```
\begin{table}[H]
\centering
\caption{JMeter experiment configuration}
\begin{tabular}{ll}
\toprule
Parameter & Configuration \\
\midrule
Target server & 103.94.135.91:8080 \\
Protocol & HTTP \\
Low-load profile & 50 threads \\
High-load profile & 100 threads \\
Ramp-up period & 100 s \\
Loop count & 1 \\
Duration threshold & 200 ms \\
HTTP implementation & HttpClient4 \\
\bottomrule
\end{tabular}
\end{table}
```

Obviously replace:

200 ms

if your final calibrated threshold differs.

## 50. Explain your Duration Assertion threshold

This should not read:

*We used 200 ms because sir said so.*

Instead write the logic:

*A duration threshold of (T) ms was selected after preliminary validation. The threshold was chosen to produce both passing and failing samples, allowing the effect of response-time violations on JMeter's reported error rate to be observed. The same threshold was retained for both workload profiles to maintain comparability.*

Then explain:

$$t \leq T \Rightarrow \text{assertion passes}$$
$$t > T \Rightarrow \text{assertion fails.}$$

JMeter marks such responses failed even if the HTTP transaction itself returned successfully.

([Apache JMeter](https://jmeter.apache.org/usermanual/component_reference) ([https://jmeter.apache.org/usermanual/component\\_reference](https://jmeter.apache.org/usermanual/component_reference)))

That's excellent material for Section 6.

---

## 51. Experimental Methodology section

Explain your experiment in chronological order:

1. Browser HTTP traffic was routed through the JMeter recorder.
2. Requests corresponding to the five target endpoints were automatically captured.
3. Irrelevant browser-generated resource requests were removed.
4. The recorded samplers were organized under separate Recording Controllers.
5. A one-thread validation run verified request correctness.
6. Duration Assertions were applied to all five HTTP samplers.
7. The 50-thread workload was executed with a 100-second ramp-up.
8. Raw samples and aggregate metrics were exported.
9. The same test plan was repeated using 100 threads while all other parameters remained unchanged.
10. HTML dashboard reports were generated from the raw CSV files.

This is the scientific-method part of your report.

---

## 52. Results section: don't interpret yet

First present data.

## Table 1

Execution time + throughput + error.

## Table 2

Bandwidth.

Then perhaps include two or three useful figures:

```
Response Times Over Time
Bytes Throughput Over Time
Active Threads Over Time
Errors
```

Don't dump 15 dashboard graphs into your PDF.

Choose graphs that support something you're actually discussing.

---

## 53. Analysis section: interpret after presenting results

For each endpoint answer:

```
What changed?

How much did it change?

Why might that change occur?

Is the explanation consistent with the documented endpoint behavior?

Was throughput affected?

Did assertion failures increase?
```

Good wording:

*Average response time for `/courses` increased from (x) ms under the 50-thread profile to (y) ms under the 100-thread profile, corresponding to an increase of (z%). This observation is consistent with the endpoint's documented CPU-intensive behavior under increasing load.*

Bad wording:

*The server CPU couldn't handle 100 users.*

You didn't actually measure CPU utilization.

Be precise about what your experiment establishes.

---

## 54. Assertion/Error section

Suppose your results are:

```
50 users:
Error = 18%

100 users:
Error = 31%
```

You should investigate whether these were:

```
HTTP/application failures

or

Duration Assertion failures
```

Then write something like:

*Inspection of failed samples showed that many requests returned valid HTTP responses but exceeded the configured (T)-ms duration threshold. Consequently, JMeter classified these samples as failures. Therefore, the reported Error % should be interpreted as a combination of transport/application failures and assertion-based performance violations rather than purely HTTP errors.*

That is a much stronger analysis.

---

## 55. Include a Limitations section

This will make the report more scientifically honest.

You have at least three limitations.

### Shared server

The assignment itself says the environment is shared across the class.

Therefore other students may be generating traffic during your experiment.

You can say:

*Because the LoadLab instance was shared among students, uncontrolled concurrent traffic may have influenced response times and throughput.*

### Ramp-up

100 threads over 100 seconds does not mean all 100 were active simultaneously throughout the experiment.

Explain that JMeter gradually started them.

### Limited repetitions

If you perform only one official run per configuration, your results don't estimate run-to-run variance very well.

You can acknowledge that without performing extra forbidden/high-load runs against the

shared server.

---

## 56. Conclusion

Keep the conclusion short.

Answer:

```
What happened to latency?  
  
What happened to throughput?  
  
Which endpoint was most load-sensitive?  
  
How did assertion failures change?  
  
What did the bandwidth endpoint show?
```

Do not introduce new numbers here.

Summarize numbers already presented.

---

## 57. A good LaTeX starting point

You can build your report around this:

```
\documentclass[11pt,a4paper]{article}

\usepackage[margin=1in]{geometry}
\usepackage{graphicx}
\usepackage{booktabs}
\usepackage{tabularx}
\usepackage{float}
\usepackage{amsmath}
\usepackage{siunitx}
\usepackage[hidelinks]{hyperref}
\usepackage{microtype}

\title{
    \textbf{Performance and Load Testing Using Apache JMeter}\\
    \large CSE214 Assessment
}

\author{
    Your Name\\
    Student ID: 2205001
}

\date{\today}

\begin{document}

\maketitle

\section*{Video Demonstration}

The complete experiment demonstration is available at:

\url{https://youtu.be/YOUR_VIDEO_LINK}

\tableofcontents
\newpage

\section{Introduction}

\section{Experimental Setup}

\subsection{Target Application}

\subsection{Target Endpoints}

\begin{table}[H]
\centering
\caption{Target HTTP endpoints}
\begin{tabular}{lll}
```

```

\toprule
Endpoint & Method & Purpose \\
\midrule
/ & GET & Baseline request \\
/notices & GET & Delayed dynamic request \\
/courses & GET & CPU-intensive request \\
/login & POST & Authentication and session handling \\
/api/download/256 & GET & Bandwidth measurement \\
\bottomrule
\end{tabular}
\end{table}

\subsection{Load Profiles}

\begin{table}[H]
\centering
\caption{Load test configuration}
\begin{tabular}{lll}
\toprule
Profile & Threads & Ramp-Up \\
\midrule
Low Load & 50 & 100 s \\
High Load & 100 & 100 s \\
\bottomrule
\end{tabular}
\end{table}

\subsection{Duration Assertion}

The maximum permitted response duration was configured as
 $(T = \text{SI}\{200\}\text{milli}\text{second})$ .

A request satisfies the performance requirement when


$$t \leq T,$$


and fails the Duration Assertion when


$$t > T.$$


\section{Methodology}

\subsection{HTTP Recording}
\subsection{Functional Validation}
\subsection{50-Thread Experiment}
\subsection{100-Thread Experiment}

```

```
\subsection{Data Collection}

\section{Experimental Results}

\subsection{Execution Time and Throughput}

% Your main results table here.

\subsection{Bandwidth}

% Received KB/s and Sent KB/s table.

\subsection{Assertion and Error Results}

\section{Performance Analysis}

\subsection{Home Page}
\subsection{Notice Board}
\subsection{Course Catalogue}
\subsection{User Authentication}
\subsection{Bandwidth Download}

\section{Duration Assertion Analysis}

\section{Limitations}

\section{Conclusion}

\appendix

\section{JMeter Test Plan}

\begin{figure}[H]
\centering
\includegraphics[width=0.8\textwidth]{figures/test-plan-tree.png}
\caption{Final Apache JMeter test plan configuration.}
\end{figure}

\end{document}
```

Replace `200 ms` with your actual final assertion threshold.

## 58. Don't overinvest in the report

This is important strategically.

Your marking scheme is:

|                     |     |
|---------------------|-----|
| Video Demonstration | 60% |
| Test Plan Design    | 15% |
| Duration Assertion  | 10% |
| Test Artifacts      | 10% |
| Analysis and Report | 5%  |

So don't turn the PDF into a 30-page research paper.

A clean report around **7–12 pages including figures/tables** would already give you plenty of room.

The 60% video deserves considerably more attention.

---

## 59. What your video should show

Since it's worth 60%, prepare your JMeter file before recording.

The assignment explicitly requires webcam picture-in-picture throughout, recorder/proxy configuration, both test profiles, assertion behavior, and HTML report generation.

Your presentation sequence should essentially be:

1. Briefly introduce the assignment.
2. Show the final JMeter tree.
3. Show HTTP Request Defaults.
4. Show Cookie Manager.
5. Show all five Recording Controllers.
6. Open HTTP(S) Test Script Recorder.
7. Explain browser proxy 127.0.0.1:8888.
8. Demonstrate one browser interaction being recorded.
9. Show the five final HTTP requests.
10. Open POST /login and show its parameters.
11. Show Duration Assertion and your threshold.
12. Show a passing sample.
13. Show an assertion-failed sample.
14. Show the 50-thread configuration.
15. Run/show the 50-thread test.
16. Show Aggregate/Summary results.
17. Show the 100-thread configuration.
18. Run/show the 100-thread test.
19. Compare important metrics.
20. Show the raw CSV.
21. Tools → Generate HTML Report.
22. Open the generated dashboard.
23. Briefly show your PDF report.

That follows the rubric almost one-to-one.

---

## 60. Final submission package

The assignment says to submit a single ZIP containing the required project artifacts.

I would prepare:

```
2205001_CSE214_Assignment/  
|  
├─ 2205001_Assessment.pdf  
|  
├─ 2205001_testPlan.jmx  
|  
├─ 2205001_result_50.csv  
|  
├─ 2205001_result_100.csv  
|  
└─ 2205001_html.zip
```

Then compress it as:

```
2205001_CSE214_Assignment.zip
```

Your PDF contains the YouTube URL, so you do not need a separate `.txt` file unless you want one for convenience.

---

## 61. The complete workflow condensed

This is the sequence I recommend you actually follow at your computer:

```
CREATE PROJECT FOLDERS
    ↓
START BLANK JMETER PLAN
    ↓
SAVE .JMX
    ↓
CREATE 50-THREAD GROUP
    ↓
ADD HTTP REQUEST DEFAULTS
    ↓
ADD COOKIE MANAGER
    ↓
ADD FIVE RECORDING CONTROLLERS
    ↓
ADD HTTP(S) RECORDER
    ↓
ADD REQUIRED LISTENERS
    ↓
CONFIGURE FIREFOX PROXY
    ↓
START RECORDER
    ↓
RECORD /
    ↓
RECORD /notices
    ↓
RECORD /courses
    ↓
SUBMIT LOGIN FORM → RECORD POST /login
    ↓
RECORD /api/download/256
    ↓
STOP RECORDER
    ↓
RESET BROWSER PROXY
    ↓
DELETE IRRELEVANT RECORDED REQUESTS
    ↓
RENAME/CLEAN FIVE SAMPLERS
    ↓
1-THREAD FUNCTIONAL TEST
    ↓
VERIFY LOGIN / COOKIES / 303
    ↓
ADD DURATION ASSERTIONS
    ↓
CALIBRATE THRESHOLD
    ↓
RESTORE 50 / 100 / 1
```

```

↓
DUPLICATE → 100-THREAD GROUP
↓
DISABLE VIEW RESULTS TREE
↓
ENABLE LOW / DISABLE HIGH
↓
CLEAR OLD DATA
↓
RUN 50
↓
SAVE RAW CSV
↓
SAVE AGGREGATE TABLE
↓
GENERATE 50 HTML DASHBOARD
↓
DISABLE LOW / ENABLE HIGH
↓
CLEAR OLD DATA
↓
RUN 100
↓
SAVE RAW CSV
↓
SAVE AGGREGATE TABLE
↓
GENERATE 100 HTML DASHBOARD
↓
FILL REPORT TABLES
↓
CALCULATE % CHANGES
↓
WRITE ANALYSIS
↓
COMPILE PDF
↓
REHEARSE VIDEO
↓
RECORD VIDEO WITH WEBCAM
↓
UPLOAD YOUTUBE UNLISTED
↓
INSERT URL INTO LATEX
↓
RECOMPILE FINAL PDF
↓
VERIFY JMX / CSV / HTML / PDF
↓
CREATE FINAL ZIP

```

---

The **next concrete thing I would do at your desk is stop thinking about the report entirely and build through Step 21: get all five requests working correctly under a one-thread validation run.** Once that works, your experiment is structurally sound; everything afterward is measurement, export, and analysis.